

STL容器用法总结

2023302121009 刘鼎琨 4.7

一、总论

1.1 三个重要概念

1. 容器：数据结构，如vector、list、deque、set、map等
2. 算法：如sort、find、copy、for_each等
3. 迭代器：连接容器、算法

1.2 自定义STL元素

```
for (vector<Person>::iterator it = v.begin(); it != v.end(); it++)
{
    cout << "Name:" << (*it).mName << " Age:" << (*it).mAge << endl;
}
```

1.3 嵌套STL

```
vector< vector<int> > v;
for (vector<vector<int>>::iterator it = v.begin(); it != v.end(); it++)
{
    for (vector<int>::iterator vit = (*it).begin(); vit != (*it).end(); vit++)
    {
        cout << *vit << " ";
    }
    cout << endl;
}
```

二、string容器

```
#include <string>
```

2.1 创建与赋值

```
char a[] = "ahello!";
string b = "bhello!";
string c(b);           //bhello!
string d(b, 1, 6);     //hello!
string e(d, 2);        //llo
string f(4, 'w');       //www
string g("happy");     //happy
string h;
h.assign("hello c++"); //hello c++
```

2.2 非质变操作

```
string example = "I like lilamama!"; //字符数组只能使用部分函数, 建议直接string
```

```
if(!example.empty()) cout << "Not empty!" << endl;

cout << example.length() << endl;    // 16
cout << example.size() << endl;      // 16

cout << example.front() << endl;      //I
cout << example.back() << endl;       //!

string cmp = "u like gay!";
cout << example.compare(cmp) << endl; // < -1

cout << example.find("li", 0) << endl; // 2
cout << example.find("li", 3) << endl; // 7
cout << example.rfind("li",5) << endl; //正数五个开始向前, 2

cout << example.at(0) << endl;        // I
```

2.3 迭代器操作

这部分可以用[]直接代替迭代器, 仅做演示.

```

string example = "I like lilama";    //字符数组只能使用部分函数，建议直接使用string

string::reverse_iterator it = example.rbegin();    //迭代器理解为指针

for (; it < example.rend(); it++)
{
    cout << (*it) << " ";           //a m a l i l   e k i l   I
}
cout << endl;

```

2.4 质变操作

```

string example = "I like lilama";    //字符数组只能使用部分函数，建议直接使用string

example.pop_back();
example.append("ama");                // 还可以用“+=”
example.push_back('!');
cout << example << endl;             // I like lilamama!

example.erase(15);                    // 删除15之后的部分
example.erase(2, 5);                  // 第二个参数为删除字符个数
//string::iterator t = example.begin();
// example.erase(t + 2, t + 7);        // 此处第二个参数为iterator end，能达到相同效果
cout << example << endl;             // I lilamama

string in = "am ";
example.insert(2, in);                 // 在第二个位置插入
//example.insert(t, char);
cout << example << endl;             // I am lilamama

example.replace(5, 8, "gay!");
//example.replace(it begin, it end, string);
cout << example << endl;             // I am gay!

example.erase();                      // 也可以用example.clear()
cout << example << endl;             // example 为空

```

2.5 string的一些补充用法

```
double b = 6.66;
    string receive;
    receive = to_string(b);
    receive += "hahaha";
    cout << receive << endl;    //6.660000hahaha

string trans = "22.2";
    double get = stod(trans);    // stoi, stof, stod
    cout << get + 10.0 << endl; //32.2
```

2.6 相关的algorithm算法

```
string a = "ldk";
    string::iterator r = a.begin();
    reverse(r, r + 2);
    cout << a << endl;    // dlk
```

2.7 Cpp输入方式汇总

1. cin标准输入流

- cin>>会自动过滤掉不可见字符（如空格 回车 tab等）；
- cin的结束符，结束后，结束符还在缓存区，也就是说cin不会去除掉行末的换行、变量间的空格。
- 例如以下代码是可以正常工作的：

```
char a,b;
cin >> a;
cout << a <<endl;
b = getchar();
cout << b << "这个输出前面有一个空格" << endl;
```

2. cin.get()

- 从输入流中读入一个字符
- 不会跳过空格、制表符、回车等特殊字符，所有的字符都能被读入

1. cin.get(字符变量名)，用来接收字符，只获取一个字符，可以接收空格，遇回车结束
2. cin.get(数组名，接收字符数目)，不支持字符串，可以接收空格，遇回车结束。

3. `cin.get()`, 主要用于舍弃输入流中不需要的字符, 或者舍弃回车, 即舍弃输入流中的一个字符.

3. `cin.getline()`

- 从输入流中读入一个字符数组
- 两种重载方式如下:

```
cin.getline(char* buf, int bufSize);
cin.getline(char* buf, int bufSize, char delim);
```

4. `getline()`

- 需要`#include`
- 和`cin.getline()`基本一致, 区别在于对象一个是字符数组, 一个是字符串.
- `cin.getline()`会增加一个`'\0'`作为结束符, 而`getline()`不会.
- 两种重载方式如下:

```
getline(input_stream, str);
getline(input_stream, str, delimiter);
```

5. `getchar()`

- 以字符为单位对输入的数据进行读取.
- 和`cin`比较像, 都不会清理回车符.

三、vector容器

```
#include <vector>
```

3.1 创建与赋值

```
vector<int> a;
vector<int> b(100); // 设置默认初始化为0的100个节点
vector<double> c(10, 1.23); // 顺序是10个1.23
int in[4] = {1, 2, 3, 4};
vector<int> d(in+1, in+4); // 2 3 4
vector<int> e(d.begin() + 1, d.end()); // 3 4
```

3.2 非质变操作

```
int a[10] = {3, 1, 4, 1, 5, 9, 2, 6};
vector<int> example(a, a + 8);

if(!example.empty()) cout << "Not empty!" << endl;

cout << example.size() << endl;           //8
cout << example.capacity() << endl;       //8 >= 8

cout << example.front() << endl;          //3
cout << example.back() << endl;           //6
```

3.3 迭代器操作

这部分可以用[]直接代替迭代器，仅做演示。

```
int a[10] = {3, 1, 4, 1, 5, 9, 2, 6};
vector<int> example(a, a + 8);

vector<int>::reverse_iterator it = example.rbegin();    //迭代器理解为指针

for (; it < example.rend(); it++)
{
    cout << (*it) << " ";                //6 2 9 5 1 4 1 3
}
cout << endl;
```

3.4 质变操作

提前写好print函数：

```

template <typename T>
void print(vector<T> &a)
{
    typename vector<T> ::iterator t;
    for (t = a.begin(); t != a.end(); ++t)
    {
        cout << (*t) << " ";
    }
    cout << endl;
}

```

//如果是const vector<T> &a, 要用const_iterator

```

int a[10] = {3, 1, 4, 1, 5, 9, 2, 6};
vector<int> example(a, a + 8);

example.reserve(1000);
example.pop_back();
example.push_back(6);
print(example); // 3 1 4 1 5 9 2 6
cout << example.capacity() << endl; // 1000, 已知容量可以提前扩容

example.resize(5);
example.resize(10);
print(example); // 3 1 4 1 5 0 0 0 0 0
cout << example.size() << endl; // 10
cout << example.capacity() << endl; // 1000, 要分清楚size 和capacity

vector<int>::iterator t = example.begin();
example.erase(t + 5, t + 10); // 第二个参数为iterator end, vector 只允许迭代器.
print(example); // 3 1 4 1 5
example.erase(t + 2); // 删除一个节点的方案
print(example); // 3 1 1 5

example.insert(t + 2, 4);
print(example); // 3 1 4 1 5

example.clear(); // vector不能用erase
print(example); // example 为空

```

3.5 与vector有关的排序算法sort() -综合案例

默认类型的情况：

```
vector<int> *tmp = new vector<int>;
tmp->push_back(1);
tmp->push_back(2);
tmp->push_back(-1);
tmp->push_back(-5);
print(*tmp);                // 1 2 -1

vector<int>::iterator it = tmp->begin();
sort(it, it + 3);
print(*tmp);                // -1 1 2 -5

sort(tmp->begin(),tmp->end(), greater<int>());
print(*tmp);                // 2 1 -1 -5

sort(tmp->begin(),tmp->end(), less<int>());
print(*tmp);                // -5 -1 1 2
```

自定义类型的比较：


```

// 定义一个结构体
struct Person {
    string name;
    int age;
};

// 定义一个比较函数，用于比较两个 Person 结构体对象的 age 成员
bool compareAge(const Person &a, const Person &b) {
    return a.age < b.age; // 按照年龄升序排序
}

int main() {
    // 创建一个存储 Person 结构体对象的 vector
    vector<Person> people = {
        {"Alice", 25},
        {"Bob", 30},
        {"Charlie", 20}
    };

    // 使用 sort() 函数对 people 中的对象按照年龄升序排序
    sort(people.begin(), people.end(), compareAge);

    // 输出排序后的结果
    for (const auto &person : people) {
        cout << "Name: " << person.name << ", Age: " << person.age << endl;
    }

    return 0;
}

```

教材上还有利用重载<的方法求解.

四、list容器

```
#include <list>
```

4.1 创建与赋值

```
list<int> a;  
list<int> b(100);           // 设置默认初始化为0的100个节点  
list<double> c(10, 1.23);  // 顺序是10个1.23  
int in[4] = {1, 2, 3, 4};  
list<int> d(in+1, in+4);    // 2 3 4
```

4.2 非质变操作

```
int a[10] = {3, 1, 4, 1, 5, 9, 2, 6};  
list<int> example(a, a + 8);  
  
if(!example.empty()) cout << "Not empty!" << endl;  
  
cout << example.size() << endl;           //8, 不存在capacity的概念  
  
cout << example.front() << endl;          //3  
cout << example.back() << endl;           //6
```

4.3 迭代器操作

基本思路和先前一致，注意插入操作和删除操作都不会造成原有list迭代器的失效.

4.4 质变操作

提前写好print函数，和先前形式一致.

```

int a[10] = {3, 1, 4, 1, 5, 9, 2, 6};
list<int> example(a, a + 8);

example.pop_back();
example.push_back(6);
print(example);                // 3 1 4 1 5 9 2 6
example.pop_front();
example.push_front(3);
print(example);                // 3 1 4 1 5 9 2 6

example.remove_if(cmp);
print (example);               // 1 1 2
//list不具备随机访问的性质, 类内部有相关函数: example.remove_if(isEven);
//vector在algorithm中有 vec.erase(remove_if(vec.begin(), vec.end(), isEven), vec.end());
// 还有example.remove(int num) 删除任一元素

example.unique();
print (example);               // 1 2

example.sort(sorting);
print (example);               // 2 1

example.insert(example.begin(), 2, 3); //list 的迭代器属于双向迭代器!
print (example);               // 3 3 2 1

example.reverse();
print (example);               // 1 2 3 3

list<int>::iterator it1 = next(example.begin(), 1);
list<int>::iterator it2 = next(example.begin(), 2);
example.erase(it1, it2);       // 只允许迭代器.
print(example);                // 1 3 3

example.clear();                // list不能用erase
print(example);                // example 为空

```

五、queue容器

```
#include <queue>
```

本身没有迭代器, 包含的函数如下:

```
queue<int> example;
example.empty();
example.size();
example.front();    //这里指的是队头的元素
example.back();
example.push(int);
example.pop();
```

六、stack容器

```
#include <stack>
```

本身没有迭代器，包含的函数如下：

```
stack<int> example;
example.empty();
example.size();
example.top();
example.push(int);
example.pop();
```

七、deque容器

```
#include<deque>
```

7.1 常用函数

这是一个双端队列，具有迭代器。原理是用一个中控器，维护每段缓冲区中的内容，缓冲区中存放真实数据。常用函数如下：

```
deque<int> example;
example.empty();
example.size();
example.front();
example.back();
example.push_front();
example.push_back();
example.pop_front(int);
example.pop_back();
example.clear();
example.begin(); example.end();
example.rbegin(); example.rend();
example.erase(example.begin());
```

7.2 推广到优先队列

```
#include<queue>
```

原理：

- 大根堆：元素值大的，先出队，系统默认这个。
- 小根堆：元素值小的，先出对。
- 优先队列定义如下，其中后两个参数选填：

```
priority_queue<int, vector<int>, less<int>> big_heap;
priority_queue<int, vector<int>, greater<int>> small_heap;
```

涉及到相关的比较，一个比较复杂的实现如下：

```

#include <iostream>
#include <queue>
using namespace std;

struct node
{
    int data;
    int grade;
};

class mycmp
{
public:
    bool operator()(const node &a, const node &b) const
    {
        return a.data < b.data;
    }
};

int main()
{
    priority_queue<node, vector<node>, mycmp> example;
    for (int i = 0; i < 10; ++ i)
    {
        node* newone = new node;
        newone->data = i;
        newone->grade = 100;
        example.push(*newone);
    }

    for (int i = 0; i < 10; ++ i)
    {
        node a = example.top();
        cout << a.data << " "; //9 8 7 6 5 4 3 2 1 0
        example.pop();
    }
    cout << endl;

    return 0;
}

```

其中包含的成员函数十分有限：

```
example.empty();
example.size();
example.push(int);
example.top();
example.pop();
```

八、常见容器的比较分析

8.1 共用的一些成员函数

```
empty()
size()
resize()
front()
back()
begin()      //迭代器
end()        //迭代器
rbegin()     //迭代器 reverse_iterator
rend()       //迭代器
erase()      //只有string允许不用迭代器
insert()     //只有string允许不用迭代器
clear()
push_back()
pop_back()
pop_front()  //只有list容器有此成员函数
push_front() //只有list容器有此成员函数
```

队、栈都是其中的特殊情况，逐一说明：

1. deque: 功能最多，以上功能都具有，此外后面大部分vector和list具有的成员函数也都可以使用.
2. queue: 质变操作只有push()和pop(), 不允许使用迭代器.
3. stack: 功能比queue还少，甚至不能读取front()和back(),只有top().
4. priority_queue: 成员函数上和stack相同。

8.2 string特有的一些成员函数

```
length()                //等价于size()
append()
find(str, place)        //还有rfind(str, place)
compare(cmp)
replace(p1, p2, str)
reverse(it, it + 2)     //#include<algorithm>
                        substr(p1, p2);
```

8.3 vector特有的一些成员函数

```
capacity()
reserve()
    swap(vec)            //vector<int>(v).swap(v);    匿名对象可以消除多余内存占用
erase(remove_if(vec.begin(), vec.end(), isEven), vec.end());
sort(tmp->begin(), tmp->end(), greater<int>());    //#include<algorithm>
```

8.4 list特有的一些成员函数

```
remove_if(cmp)
sort(cmp)
remove()
unique()
reverse()
```